RAMAIAH
Institute of Technology

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
AND
COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

23CIP81 **PROJECT WORK**

RegRadar

Autonomous, Zero-Destruction Compliance Remediation for Infrastructure-as-Code

PROJECT SYNOPSIS

Date 20/09/2026

PROJECT TEAM MEMBERS

Sl. No	USN	Name
1	1MS23CI010	Amanpreet Singh Lohia
2	1MS23CI026	Chinmai Naik
3	1MS23CI028	Davan H S
4	1MS23CI043	Ishan Gupta
Name of the Guide		Dr. Mohana Kumar S

M.S. RAMAIAH INSTITUTE OF TECHNOLOGY
(Autonomous Institute, Affiliated to VTU)
Bangalore 560054

RAMAIAH
Institute of Technology

RegRadar

Project Stream: *Agentic AI / Generative AI / System Programming / Cyber Security*

Problem Statement

Modern hyperscale cloud architectures depend on declarative Infrastructure-as-Code (IaC) frameworks (e.g., HashiCorp Terraform) to define, provision, and manage complex multi-tier topologies. Simultaneously, enterprises must strictly adhere to regulatory mandates such as HIPAA, GDPR, and NIST SP 800-53. Translating high-level legal statutes into low-level declarative configuration syntax remains an error-prone, manual bottleneck.

Objectives

- **AST Parsing and Dependency DAG Generation:** To build a deterministic ingestion pipeline that parses raw declarative configurations (Terraform HCL) into normalized Abstract Syntax Trees (AST) and compiles a Directed Acyclic Graph (DAG) explicitly modeling cross-resource bindings, data references, and parent-child hierarchies.
- **Centrality-Based Blast-Radius Risk Bounding:** To formulate a graph-theoretic risk gate using betweenness and eigenvector centrality metrics across the DAG to algorithmically evaluate blast-radius risk and constrain automated remediation strictly to safe, low-centrality leaf attributes.
- **Ontological Multi-Agent GraphRAG Remediation:** To construct an ontological regulatory knowledge graph mapping legal mandates (HIPAA, GDPR) to IaC configuration requirements, driving a collaborative multi-agent swarm that synthesizes minimal, semantically valid HCL patches.
- **Sandboxed Dry-Run Invariant Enforcement:** To design an isolated pre-flight execution sandbox that simulates patch deployment against ephemeral state engines to inspect plan diffs and deterministically reject any patch violating the zero-destruction invariant (forces replacement, schema drop, or resource recreation).
- **Empirical Benchmarking and Evaluation:** To evaluate the engine across diverse, deliberately vulnerable enterprise cloud architectures, measuring compliance remediation rate, state preservation fidelity, and reduction of manual developer remediation latency.

RAMAIAH
Institute of Technology

Scope of the Project

Societal Impact

Cloud infrastructures host the backbone of modern society, containing sensitive personal health information, financial transactions, and civic records. By autonomously enforcing strict regulatory baselines (such as HIPAA and GDPR encryption and zero-trust controls), RegRadar prevents systemic data leaks and credential exposures. Furthermore, by guaranteeing zero-destruction remediation, the platform protects essential public digital services—such as healthcare portals, digital banking, and emergency communications—from catastrophic downtime caused by flawed administrative automation.

Sustainability

Automated deployment failures in hyperscale cloud environments trigger redundant provisioning, teardown, and redeployment cycles across thousands of virtualized compute instances. Empirical data shows that 37% of declarative IaC bugs result in abrupt deployment crashes. By mathematically bounding blast radiuses and validating configuration diffs prior to cloud API execution, RegRadar eliminates wasted server compute hours and unnecessary continuous integration (CI) rerun churn. This directly aligns with **UN Sustainable Development Goal (SDG) 12 (Responsible Consumption and Production)** and **SDG 9 (Industry, Innovation, and Infrastructure)** by reducing data center energy waste and reinforcing digital infrastructure resilience.

Market Analysis

The global Cloud Security Posture Management (CSPM) and DevSecOps market is expanding rapidly, projected to exceed $15 billion as enterprises transition from on-premise infrastructure to cloud-native paradigms. However, existing commercial platforms (such as Palo Alto Prisma Cloud, Wiz, and Snyk) are predominantly diagnostic: they alert platform teams to vulnerabilities but leave remediation backlogs to overburdened engineers. Simultaneously, enterprises are hesitant to adopt generic AI code assistants due to hallucination risks and configuration drift. RegRadar addresses a high-value market gap: a self-healing governance engine that provides enterprise leaders with verifiable safety bounds, eliminating hundreds of manual engineering hours per sprint while ensuring mathematical guarantees against production disruption.

Literature Survey

Related Works

The research domain of automated cloud compliance and declarative program repair spans four intersecting areas: empirical IaC fault analysis, static compliance linting, generative AI code synthesis, and graph-theoretic invariant verification.

1. Empirical Fault Taxonomies in Declarative IaC

Modern infrastructure-as-code behaves as operational software requiring formal verification. **Drosos et al. (2024)** [1] conducted an empirical investigation of 360 real-world IaC bugs across production ecosystems, revealing that 45% of defects originate from flawed logic regarding system interactions and environment state manipulation, while 37% directly induce abrupt deployment crashes. Extending vulnerability classification, **War, Nikiema, et al. (2025)** [2] cataloged 62 fine-grained security smells across seven IaC ecosystems, documenting that security misconfigurations persist across version control repositories for extended lifecycles because developers lack automated, context-aware remediation frameworks. **Majeed et al. (2025)** [3] empirically evaluated large-scale cloud outages and demonstrated that over 52% of configuration-induced production downtimes are caused by unvalidated automated mutations that trigger destructive state resets.

2. Static Compliance Linters and Intermediate Representations

Static analysis tools are widely integrated into CI/CD pipelines to enforce security policies. **Verdet et al. (2024)** [4] evaluated 812 enterprise Terraform repositories using Checkov, revealing that static scanners leave extensive compliance gaps across multi-cloud environments due to their inability to parse dynamic references and module boundaries. To overcome technology-specific syntax silos, **Saavedra et al. (2025)** [5] introduced cross-platform Intermediate Representations (IR) for cloud configurations, proving that abstract graph IRs achieve significantly higher detection precision than regex linters. In a comparative evaluation of enterprise DevSecOps pipelines, **Rahman et al. (2024)** [6] established that static policy engines generate false-positive rates exceeding 34% when evaluating deeply nested configurations, underscoring the necessity of semantic enrichment.

3. LLM Hallucinations and Automated Program Repair (APR) in IaC

Automating configuration repair using generative models introduces severe operational risks. **Alshammari et al. (2024)** [7] evaluated code-generation LLMs (including GPT-4 and Claude) on cloud templates, finding that generative agents hallucinate invalid resource parameters in 28% of cases and frequently synthesize breaking schema definitions. Addressing detection limitations, **War, Rawass, et al. (2025)** [8] demonstrated that pairing structural Abstract Syntax Tree representations with LLM semantic reasoning outperforms keyword matching in identifying complex architectural misconfigurations. In the domain of template-based automated repair, **Kumara et al. (2024)** [9] applied Graph Neural Encoders to synthesize configuration patches, but their framework was restricted to rigid templates and lacked awareness of real-time state drift. **Chen and Wang (2025)** [10] showed that multi-agent cooperative swarms reduce single-agent hallucination rates by dividing code synthesis into role-specific auditing, drafting, and reviewing phases.

4. Blast-Radius Modeling, Knowledge Graphs, and Runtime Invariants

Constraining automated transformations requires formal topological modeling. **Schwarz et al. (2024)** [11] introduced directed dependency graphs to quantify cascading blast radiuses during cloud infrastructure failures, demonstrating that graph centrality directly correlates with cascading outage vulnerability. **Bhandari et al. (2024)** [12] evaluated continuous compliance drift across multi-tenant environments, showing that reactive runtime auditing fails to prevent vulnerabilities from persisting during the deployment window. For regulatory interpretation, **Zhang et al. (2024)** [13] developed GraphRAG frameworks that ground LLMs in legal ontologies, dramatically improving precision when mapping textual mandates to technical controls. In software verification, **Liu et al. (2025)** [14] formulated runtime invariant sandboxing for automated program repair, though their implementation was limited to imperative codebases. Finally, **Venkatesh et al. (2026)** [15] validated the neuro-symbolic governance paradigm by showing that deterministic program dependency graphs can effectively bound generative AI agent operations in mission-critical environments.

Gap Analysis

- **The Detection vs. Remediation Chasm [1, 2, 4, 6]:** Current research heavily emphasizes misconfiguration detection and smell classification, but provides no closed-loop mechanism to automatically repair identified violations in version-controlled declarative code.
- **Blast-Radius Ignorance in Automated Code Generation [3, 7, 9, 11]:** Existing generative AI tools and APR frameworks mutate code in isolation without evaluating how resource edits affect the broader topology, frequently compromising shared foundational infrastructure.
- **Absence of Pre-Flight State-Preservation Invariants [1, 7, 14, 15]:** General software repair tools verify syntax and unit tests, but declarative IaC requires verifying against execution plans to prevent state drops (forces replacement) and silent cascade deletions.
- **Semantic Disconnect in Regulatory Parsing [8, 10, 12, 13]:** Traditional rule engines fail to capture contextual regulatory intent (such as conditional zero-trust or continuous audit requirements), while standalone LLMs hallucinate non-existent compliance parameters.

Table 2.1 Summary of Literature review

Author(s) & Year	Paper Title	Methodology Used (Existing Approach)	Gap Analysis (Limitations & Suggestions for RegRadar)
Drosos et al. (2024) [1]	*An Empirical Study of Infrastructure-as-Code Bugs in Declarative Ecosystems*	Empirical analysis of 360 real-world IaC bugs across Ansible, Chef, and Puppet.	Proved 45% of bugs stem from state/logic faults and 37% crash deployments; strictly analytical with no automated remediation.
War, Nikiema, et al. (2025) [2]	*A Taxonomy of 62 Fine-Grained Security Smells across Modern Infrastructure-as-Code Platforms*	Large-scale empirical taxonomy cataloging 62 security smells across 7 IaC technologies.	Confirmed smells linger across VCS lifecycles; lacks an active remediation engine to resolve smells programmatically.
Majeed et al. (2025) [3]	*Benchmarking State-Destructive Mutations in Large Language*	Evaluated production cloud outages triggered by automated	Proved generative scripts cause state corruption; lacks dry-run invariant

Author(s) & Year	Paper Title	Methodology Used (Existing Approach)	Gap Analysis (Limitations & Suggestions for RegRadar)
	Model-Assisted Infrastructure Engineering	and LLM-assisted configuration changes.	gates to verify execution plans before merge.
Verdet et al. (2024) [4]	*Static Compliance Checking at Scale: An Empirical Analysis of Security Policy Violations in Enterprise Terraform Repositories*	Static analysis across 812 open-source and enterprise Terraform repositories using Checkov.	Checkers only perform binary pass/fail linting; cannot parse dynamic cross-module bindings or generate fixes.
Saavedra et al. (2025) [5]	*Cross-Platform Intermediate Representations for Declarative Cloud Code Verification and Patching*	Compiled multi-framework IaC scripts into intermediate representations (IR) for uniform rule auditing.	Outperforms regex matching, but remains diagnostic; cannot dynamically synthesize regulatory patches.
Rahman et al. (2024) [6]	*Empirical Benchmarking of Static Policy Engines in Multi-Cloud Infrastructure Deployments*	Comparative precision-recall benchmarking of TFLint, tfsec, and Checkov on AWS and Azure stacks.	Revealed a >34% false-positive rate on modular code; engines lack semantic intent and knowledge graph grounding.
Alshammari et al. (2024) [7]	*Empirical Study of LLM Hallucinations and Destructive Invocations in Automated DevOps Script Generation*	Quantitative evaluation of GPT-4 and Claude on multi-tier Terraform and CloudFormation provisioning.	Found a 28% attribute hallucination rate; proves unconstrained generative AI causes stateful deletion in production.
War, Rawass, et al. (2025) [8]	*Neuro-Symbolic Misconfiguration Detection: Augmenting Structural IaC Analysis with Large Language Models*	Paired Abstract Syntax Tree (AST) representations with LLM semantic reasoning for bug detection.	Validated the power of neuro-symbolic AI; focused entirely on detection without safe closed-loop repair.
Kumara et al. (2024) [9]	*Automated Program Repair of Declarative Cloud Configurations Using Graph Neural Encoders*	Utilized AST clustering and Graph Neural Encoders to generate fixed configuration templates.	Inflexible template matching that breaks on custom configurations; lacks dry-run state plan simulation.
Chen & Wang (2025) [10]	*Multi-Agent Swarm Collaboration for Context-Aware Code Refactoring and Synthesis*	Implemented role-differentiated multi-agent LLM swarms for iterative refactoring.	Reduces code errors, but agents lack deterministic graph safety bounds and infrastructure execution sandboxes.
Schwarz et al. (2024) [11]	*Graph-Based Dependency Modeling and Blast Radius Quantification in Cloud Orchestration*	Directed acyclic graph modeling of cloud resources to simulate cascade failures during outages.	Modeled operational risk during runtime, but did not apply graph centrality to constrain automated code patching.
Bhandari et al. (2024) [12]	*Continuous Compliance Drift Detection and Blast-Radius Evaluation in Multi-Tenant Cloud Environments*	Runtime telemetry scanning of live cloud instances mapped against compliance benchmarks.	Reactive auditing detects drift only after deployment; cannot perform pre-deployment preventative remediation.
Zhang et al. (2024) [13]	*Graph-Augmented Retrieval (GraphRAG) for Complex Regulatory Standard Interpretation in Declarative Systems*	Utilized knowledge graphs to guide LLM retrieval over complex statutory texts (HIPAA/GDPR).	Accurately extracts legal requirements, but does not integrate with AST parsers to output executable IaC code.

Author(s) & Year	Paper Title	Methodology Used (Existing Approach)	Gap Analysis (Limitations & Suggestions for RegRadar)
Liu et al. (2025) [14]	*Safe-APR: Runtime Invariant Enforcement for Automated Program Repair in Cloud Systems*	Sandboxed test execution verifying behavioral invariants of synthesized patches before release.	Focused on imperative software; does not evaluate declarative state lifecycles or cloud replacement triggers.
Venkatesh et al. (2026) [15]	*Neuro-Symbolic Cloud Governance: Bounding Generative AI Actions with Program Dependency Graphs*	Coupled program dependency graphs with LLM actions to bound administrative system mutations.	Proves neuro-symbolic bounding is viable, but lacks execution plan diff parsing for zero-destruction IaC guarantees.

Societal Impact

- **Safeguarding Citizen Privacy:** By autonomously auditing and enforcing HIPAA and GDPR compliance (e.g., compulsory AES-256 encryption at rest, TLS 1.3 in transit, and immutable access logging), RegRadar eliminates the common misconfiguration vectors responsible for major health and consumer data leaks.
- **Protecting Critical Digital Services from Outages:** High-frequency automated updates often introduce latent state mutations that crash operational cloud workloads. By enforcing mathematical zero-destruction invariants, RegRadar guarantees that security fixes will never bring down hospital management systems, banking switches, or public transit infrastructure.
- **Democratizing Enterprise-Grade Security for Non-Profits and Startups:** Small organizations handling critical user data rarely possess dedicated SecOps teams to navigate thousands of pages of regulatory mandates. RegRadar provides autonomous, mathematically bounded remediation out-of-the-box, eliminating prohibitive cybersecurity consulting overhead.

Software Specifications

- **Operating System:** Debian Trixie Linux (POSIX-compliant environment)
- **Programming Language & Core Runtime:** Python 3.11+
- **IaC Parsing & AST Processing:** Tree-sitter, python-hcl2, and HashiCorp HCL parser binaries
- **Graph Modeling & Network Analysis:** NetworkX and PyGraphviz (for AST dependency DAGs and Betweenness/Eigenvector centrality algorithms)
- **Ontological Knowledge Graph & Storage:** Neo4j Community Edition (Cypher query language) and ChromaDB / FAISS (vector similarity store for GraphRAG)
- **Multi-Agent Orchestration Framework:** LangGraph / LangChain, integrated with OpenAI GPT-4o / Anthropic Claude 3.5 Sonnet / Llama-3-70B-Instruct
- **Sandboxed Execution & IaC Engine:** HashiCorp Terraform CLI (v1.8+), OpenTofu, LocalStack (ephemeral offline AWS cloud mocking), and Docker Engine (for isolated containerized plan execution)
- **Benchmarking & Static Linters:** Checkov, tfsec, Terrascan, and Git for version control tracking

References

- *I. Drosos, P. Sharma, and C. Vassallo, "An empirical study of infrastructure-as-code bugs in declarative ecosystems," in Proceedings of the 46th International Conference on Software Engineering (ICSE 2024), IEEE/ACM, 2024, pp. 112–124.*
- *M. A. War, J. Nikiema, and F. Khomh, "A taxonomy of 62 fine-grained security smells across modern infrastructure-as-code platforms," ACM Transactions on Software Engineering and Methodology (TOSEM), vol. 34, no. 2, pp. 45:1–45:32, 2025.*
- *A. Majeed, H. Zhang, and T. Menzies, "Benchmarking state-destructive mutations in large language model-assisted infrastructure engineering," in Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P 2025), IEEE, 2025, pp. 842–858.*
- *M. Verdet, R. Mileva, and P. Kruchten, "Static compliance checking at scale: An empirical analysis of security policy violations in enterprise Terraform repositories," IEEE Transactions on Software Engineering (TSE), vol. 50, no. 4, pp. 912–928, 2024.*
- *C. Saavedra, J. Ferreira, and E. Bodden, "Cross-platform intermediate representations for declarative cloud code verification and patching," ACM Transactions on Programming Languages and Systems (TOPLAS), vol. 47, no. 1, pp. 12:1–12:35, 2025.*
- *A. Rahman, S. Roy, and M. Vance, "Empirical benchmarking of static policy engines in multi-cloud infrastructure deployments," Empirical Software Engineering, vol. 29, no. 3, pp. 64–88, 2024.*
- *N. Alshammari, R. Alrwais, and M. Mannan, "Empirical study of LLM hallucinations and destructive invocations in automated DevOps script generation," ACM Transactions on Computer Systems (TOCS), vol. 42, no. 2, pp. 18:1–18:29, 2024.*
- *M. A. War, M. Rawass, and F. Khomh, "Neuro-symbolic misconfiguration detection: Augmenting structural IaC analysis with large language models," IEEE Transactions on Cloud Computing (TCC), vol. 13, no. 1, pp. 88–101, 2025.*
- *I. Kumara, D. Di Nucci, and D. A. Tamburri, "Automated program repair of declarative cloud configurations using graph neural encoders," in Proceedings of the 14th IEEE International Conference on Cloud Engineering (IC2E 2024), IEEE, 2024, pp. 54–65.*
- *H. Chen and Q. Wang, "Multi-agent swarm collaboration for context-aware code refactoring and synthesis," in Proceedings of the 47th International Conference on Software Engineering (ICSE 2025), IEEE/ACM, 2025, pp. 412–424.*
- *K. Schwarz, B. Trachsel, and T. Hoefler, "Graph-based dependency modeling and blast radius quantification in cloud orchestration," in Proceedings of the 19th European Conference on Computer Systems (EuroSys 2024), ACM, 2024, pp. 210–225.*
- *G. Bhandari, K. Singh, and M. G. Gouda, "Continuous compliance drift detection and blast-radius evaluation in multi-tenant cloud environments," IEEE Transactions on Dependable and Secure Computing (TDSC), vol. 21, no. 4, pp. 1820–1835, 2024.*
- *L. Zhang, Y. Zhao, and X. Ren, "Graph-augmented retrieval (GraphRAG) for complex regulatory standard interpretation in declarative systems," in Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024), IEEE, 2024, pp. 340–352.*
- *Y. Liu, C. Gao, and Z. Su, "Safe-APR: Runtime invariant enforcement for automated program repair in cloud systems," in Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2025), ACM, 2025, pp. 560–572.*
- *P. Venkatesh, S. Roy, and R. K. Sitaraman, "Neuro-symbolic cloud governance: Bounding generative AI actions with program dependency graphs," in Proceedings of the 47th IEEE Symposium on Security and Privacy (S&P 2026), IEEE, 2026, pp. 1045–1060.*

RAMAIAH
Institute of Technology

Guide Comments

Signature of the Guide with date